Interactions tactiles et gestuelles

420-249-AH - Programmation iOS

SwiftUI - Toucher, appui prolongé, déplacement, zoom et rotation

Khalil Loghlam

Interactions tactiles

SwiftUI permet de détecter plusieurs types d'interactions et de modifier l'état de l'interface en réponse aux gestes de l'utilisateur.

Toucher	Appui prolongé	Déplacement	Pincement	Rotation
`TapGesture`	`LongPressGesture`	`DragGesture`	`MagnifyGesture`	`RotateGesture`

Toucher une vue

```
Image(systemName: "heart")
    .font(.largeTitle)
    .onTapGesture {
        favori.toggle()
    }
```

Exemple visuel

```
Image(
    systemName:
        favori ? "heart.fill" : "heart"
)
.onTapGesture {
    favori.toggle()
}
```

Button ou onTapGesture ?

Pour une véritable action comme Enregistrer, Supprimer ou Continuer, on privilégie Button. onTapGesture est surtout utile lorsqu'une vue elle-même doit réagir au toucher.

Double toucher et appui prolongé

Double toucher

```
Image("photo")
    .onTapGesture(count: 2) {
        aime.toggle()
    }
```

Appui prolongé

```
Text("Maintenir")
    .onLongPressGesture(
        minimumDuration: 1
    ) {
        afficherOptions = true
    }
```

Un appui prolongé convient bien aux actions secondaires ou aux options contextuelles.

DragGesture

DragGesture permet de suivre le déplacement du doigt et d'appliquer ce déplacement à une vue.

```
@State private var position = CGSize.zero

Rectangle()
    .frame(width: 100, height: 100)
    .offset(position)
    .gesture(
        DragGesture()
            .onChanged { value in
                position = value.translation
            }
    )
```

value.translation

Représente le déplacement effectué depuis le début du geste.

onChanged et onEnded

Un geste peut produire une réaction pendant l'interaction, puis une autre lorsque le geste se termine.

```
DragGesture()
    .onChanged { value in
        position = value.translation
    }
    .onEnded { value in
        position = .zero
    }
```

- Déplacer une vue pendant le geste.
- Conserver ou réinitialiser la position à la fin.
- Déclencher une action lorsque le geste est terminé.

État temporaire avec @GestureState

Certaines valeurs ne sont utiles que pendant le geste. @GestureState est conçu pour cet état temporaire.

```
@GestureState
private var deplacement = CGSize.zero

DragGesture()
    .updating($deplacement) { value, state, _ in
        state = value.translation
    }

.offset(deplacement)
```

Avantage

La valeur revient automatiquement à son état initial lorsque le geste se termine.

Pincement avec MagnifyGesture

MagnifyGesture permet de détecter un pincement pour agrandir ou réduire une vue.

```
@State private var echelle = 1.0

Image(systemName: "photo")
    .font(.system(size: 100))
    .scaleEffect(echelle)
    .gesture(
        MagnifyGesture()
            .onChanged { value in
                echelle = value.magnification
            }
    )
```

Utilisations

- Photos
- Documents
- Contenu visuel

Rotation avec RotateGesture

RotateGesture permet de faire pivoter une vue en fonction du mouvement des doigts.

```
@State private var angle = Angle.zero

Image(systemName: "square")
    .font(.system(size: 100))
    .rotationEffect(angle)
    .gesture(
        RotateGesture()
            .onChanged { value in
                angle = value.rotation
            }
    )
```

Résultat

La vue suit l'angle de rotation détecté pendant le geste.

Combiner plusieurs gestes

Une même vue peut reconnaître plusieurs gestes. Ils peuvent être simultanés, séquentiels ou exclusifs.

Principales compositions

- simultaneously(with:) : deux gestes sont reconnus en même temps.
- sequenced(before:) : un geste doit précéder l'autre.
- exclusively(before:) : un seul des gestes doit gagner.

```
Image(systemName: "photo")
    .offset(position)
    .scaleEffect(echelle)
    .gesture(
        drag.simultaneously(
            with: zoom
        )
    )
```

Zone tactile et retour visuel

Étendre la zone tactile

```
HStack {
    Image(systemName: "star")
    Text("Favori")
    Spacer()
}
.contentShape(Rectangle())
.onTapGesture {
    favori.toggle()
}
```

Ajouter un retour visuel

```
.onTapGesture {
    withAnimation {
        selectionne.toggle()
    }
}

.scaleEffect(
    selectionne ? 1.2 : 1
)
```

Choisir le bon geste

onTapGesture

Sélectionner une vue, ouvrir une carte, gérer un favori.

MagnifyGesture

Agrandir ou réduire une photo ou un document.

onLongPressGesture

Afficher des options ou déclencher une action secondaire.

RotateGesture

Faire pivoter un objet ou un contenu visuel.

DragGesture

Déplacer un élément ou faire glisser une carte.

Button

À privilégier pour une action explicite de l'interface.

Exercice - Carte interactive

Créer une carte contenant une icône, un titre et une image, puis ajouter les interactions suivantes :

- Toucher : changer l'état favori.
- Appui prolongé : afficher un message.
- Glissement : déplacer la carte.
- Pincement : agrandir ou réduire la carte.
- Permettre le déplacement et le zoom simultanément.

Objectif

Mettre en pratique plusieurs gestes sur une même vue tout en gardant l'interface claire et naturelle.

Conclusion

SwiftUI permet d'ajouter des interactions tactiles directement aux vues et de relier les gestes à l'état de l'interface.

- onTapGesture et onLongPressGesture couvrent les interactions simples.
- DragGesture suit un déplacement.
- MagnifyGesture gère le zoom.
- RotateGesture gère la rotation.
- @GestureState convient aux valeurs temporaires d'un geste.
- Les gestes peuvent être combinés lorsqu'une interaction plus riche est nécessaire.